

ML Assignment #2

Unsupervised Learning: PCA, K-means, DBSCAN

20214455 전형원

1. 실험 환경 (Experimental Setup)

본 실험은 Google Colab 환경에서 Python을 사용하여 수행하였으며, 아래와 같은 라이브러리를 활용하였다.

항목	내용
실험 플랫폼	Google Colab
프로그래밍 언어	Python 3
딥러닝 프레임워크	PyTorch (torchvision)
머신러닝 라이브러리	scikit-learn
기타 라이브러리	NumPy, pandas, matplotlib, seaborn
하드웨어	T4 GPU + 고용량 RAM (Google Colab 제공)

2. 데이터셋 (Dataset)

Fashion MNIST는 28×28 픽셀의 흑백 패션 이미지 데이터셋으로, 10가지 카테고리로 구성되어 있다. 본 과제에서는 테스트 데이터(10,000장)만 사용하였다.

구분	내용
클래스 수	10개 (T-shirt, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot)
사용 데이터	테스트 데이터만 사용
테스트 데이터	10,000장
이미지 크기	28×28 (흑백)
입력 차원	784

▶ 데이터 로딩 코드

```
Fashion_mnist_test_transform = transforms.Compose([transforms.ToTensor()])
testset_Fashion_mnist = datasets.FashionMNIST(root='./data', train=False,
download=True,
transform=Fashion_mnist_test_transform)
FM_test = DataLoader(testset_Fashion_mnist, batch_size=32, shuffle=False,
num_workers=2)

FM_test_images = []
FM_test_labels = []
for batch in FM_test:
```

```

images, labels = batch
images_flat = images.view(images.shape[0], -1)
FM_test_images.append(images_flat.numpy())
FM_test_labels.append(labels.numpy())

FM_test_images = np.vstack(FM_test_images)
FM_test_labels = np.concatenate(FM_test_labels)

X_ = pd.DataFrame(data=FM_test_images)
y_ = pd.Series(data=FM_test_labels)

print("FM_test_images shape:", FM_test_images.shape)
print("FM_test_labels shape:", FM_test_labels.shape)

```

▶ 데이터 로딩 결과 화면

```

100%|██████████| 26.4M/26.4M [00:01<00:00, 19.3MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 303kB/s]
100%|██████████| 4.42M/4.42M [00:00<00:00, 5.63MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 20.5MB/s]
FM_test_images shape: (10000, 784)
FM_test_labels shape: (10000,)

```

3. 알고리즘 및 하이퍼파라미터 (Algorithms & Hyperparameters)

3.1 PCA (주성분 분석)

PCA(Principal Component Analysis)는 고차원 데이터를 저차원으로 차원 축소하는 기법이다. 본 실험에서는 원본 784차원 데이터를 100, 50, 10차원으로 각각 축소하여 K-means 및 DBSCAN 클러스터링에 활용하였다.

하이퍼파라미터	설정값	설명
n_components	100, 50, 10	축소할 차원 수
원본 차원	784	28×28 이미지 평탄화

▶ PCA 구현 코드

```

# 원본 784 차원
X_pca_784 = X_.copy()

# PCA 100 차원
pca_100 = PCA(n_components=100, random_state=42)
X_pca_100 = pd.DataFrame(pca_100.fit_transform(X_))

```

```
# PCA 50 차원
pca_50 = PCA(n_components=50, random_state=42)
X_pca_50 = pd.DataFrame(pca_50.fit_transform(X_))

# PCA 10 차원
pca_10 = PCA(n_components=10, random_state=42)
X_pca_10 = pd.DataFrame(pca_10.fit_transform(X_))

print("원본 차원:", X_pca_784.shape)
print("PCA 100 차원:", X_pca_100.shape)
print("PCA 50 차원:", X_pca_50.shape)
print("PCA 10 차원:", X_pca_10.shape)
```

▶ PCA 결과 화면

```
원본 차원: (10000, 784)
PCA 100차원: (10000, 100)
PCA 50차원: (10000, 50)
PCA 10차원: (10000, 10)
```

3.2 K-means 클러스터링

K-means는 데이터를 k개의 클러스터로 분할하는 알고리즘이다. 본 실험에서는 클래스 수와 동일하게 k=10으로 설정하였으며, PCA로 차원 축소된 데이터(784, 100, 50, 10차원)에 각각 적용하였다.

하이퍼파라미터	설정값	설명
n_clusters (k)	10	클러스터 수 (클래스 수와 동일)
n_init	10	초기화 반복 횟수
random_state	42	재현성을 위한 랜덤 시드
init	k-means++	초기 중심점 선택 방법
max_iter	300	최대 반복 횟수

▶ K-means 구현 코드

```
pca_data = {
    784: X_pca_784.values,
    100: X_pca_100.values,
    50: X_pca_50.values,
    10: X_pca_10.values,
}

kmeans_labels = {}
kmeans_ari = {}
```

```

for dim in [784, 100, 50, 10]:
    km = KMeans(n_clusters=10, init='k-means++', n_init=10, max_iter=300,
random_state=42)
    km.fit(pca_data[dim])
    kmeans_labels[dim] = km.labels_
    ari = metrics.adjusted_rand_score(FM_test_labels, km.labels_)
    kmeans_ari[dim] = ari
    print(f"Dim={dim:4d} | ARI={ari:.4f}")

print("\n==== K-Means ARI 결과 =====")
print(f"{'Dimension':<12} {'ARI'}")
print("-"*20)
for dim in [784, 100, 50, 10]:
    print(f"{'dim':<12} {'kmeans_ari[dim]:.4f}'")

```

3.3 DBSCAN 클러스터링

DBSCAN(Density-Based Spatial Clustering of Applications with Noise)은 밀도 기반 클러스터링 알고리즘으로, k 를 미리 지정하지 않아도 되며 노이즈 포인트를 자동으로 처리한다. 차원별로 적절한 **eps** 값을 다르게 설정하였다.

차원	eps	min_samples	설명
784	10	5	원본 차원, 거리 스케일 큼
100	4	5	PCA 100차원
50	3	5	PCA 50차원
10	1.5	5	PCA 10차원, 거리 스케일 작음

▶ DBSCAN 구현 코드

```

dbscan_params = {
    784: {'eps': 10, 'min_samples': 5},
    100: {'eps': 4, 'min_samples': 5},
    50: {'eps': 3, 'min_samples': 5},
    10: {'eps': 1.5, 'min_samples': 5},
}

dbscan_labels = {}
dbscan_ari = {}

for dim in [784, 100, 50, 10]:
    eps = dbscan_params[dim]['eps']
    min_samples = dbscan_params[dim]['min_samples']

    db = DBSCAN(eps=eps, min_samples=min_samples, n_jobs=-1)
    db.fit(pca_data[dim])
    dbscan_labels[dim] = db.labels_

n_clusters = len(set(db.labels_)) - (1 if -1 in db.labels_ else 0)

```

```

n_noise = np.sum(db.labels_ == -1)
ari = metrics.adjusted_rand_score(FM_test_labels, db.labels_)
dbscan_ari[dim] = ari
print(f"Dim={dim:4d} | eps={eps} | clusters={n_clusters} | noise={n_noise} |
ARI={ari:.4f}")

print("\n==== DBSCAN ARI 결과 =====")
print(f"{'Dimension':<12} {'Clusters':<12} {'ARI'}")
print("-"*30)
for dim in [784, 100, 50, 10]:
    n_clusters = len(set(dbscan_labels[dim])) - (1 if -1 in dbscan_labels[dim]
    else 0)
    print(f"{dim:<12} {n_clusters:<12} {dbscan_ari[dim]:.4f}")

```

4. 실험 결과 (Experimental Results)

4.1 K-means 결과

K-means 클러스터링 결과 및 ARI(Adjusted Rand Index) 점수는 아래와 같다.

차원 (Dimension)	ARI 점수
784 (원본)	0.3498
100	0.3402
50	0.3516
10	0.3497

▶ K-means ARI 결과 화면

```

Dim= 784 | ARI=0.3498
Dim= 100 | ARI=0.3402
Dim=  50 | ARI=0.3516
Dim=  10 | ARI=0.3497

==== K-Means ARI 결과 ====
Dimension      ARI
-----
784            0.3498
100            0.3402
50             0.3516
10             0.3497

```

▶ K-means t-SNE 시각화 코드

```
np.random.seed(42)
sample_idx = np.random.choice(len(FM_test_images), 100, replace=False)

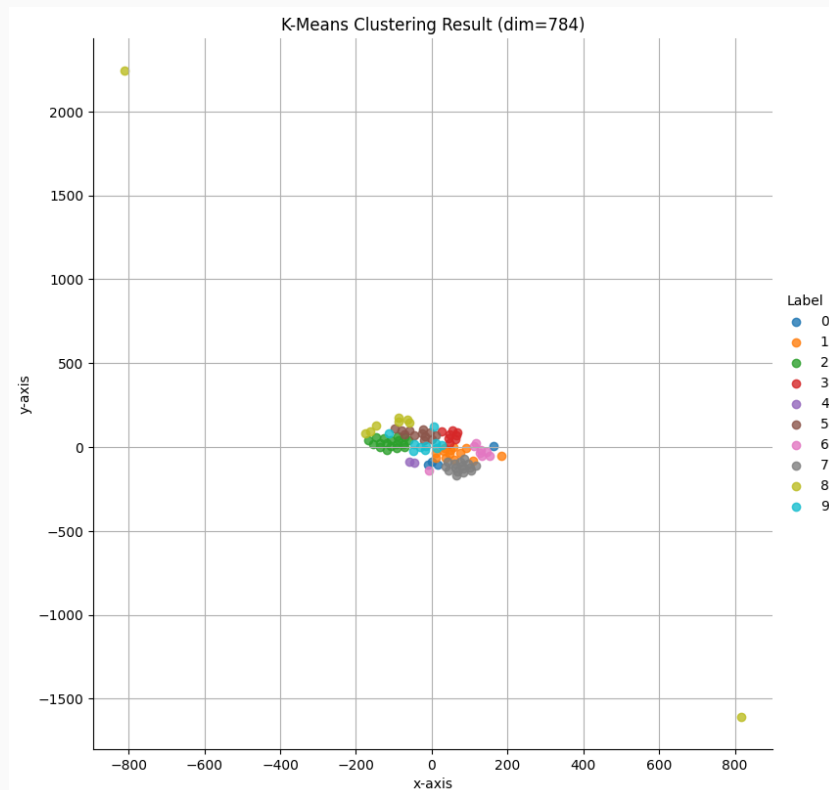
for dim in [784, 100, 50, 10]:
    # 100 개 샘플링
    X_sample = pca_data[dim][sample_idx]
    cluster_sample = kmeans_labels[dim][sample_idx]

    # t-SNE
    tsne = TSNE(n_components=2, learning_rate=300, perplexity=30,
                early_exaggeration=12, init='random', random_state=42)
    X_tsne = pd.DataFrame(tsne.fit_transform(X_sample))

    # DataFrame 구성
    testDF = pd.DataFrame(data=X_tsne.values, columns=["x-axis", "y-axis"])
    testDF = pd.concat((testDF, pd.Series(cluster_sample, name="Label")), axis=1)

    # 시각화
    sns.lmplot(x="x-axis", y="y-axis", hue="Label", data=testDF, fit_reg=False,
               height=8)
    plt.title(f"K-Means Clustering Result (dim={dim})")
    plt.grid()
    plt.show()
```

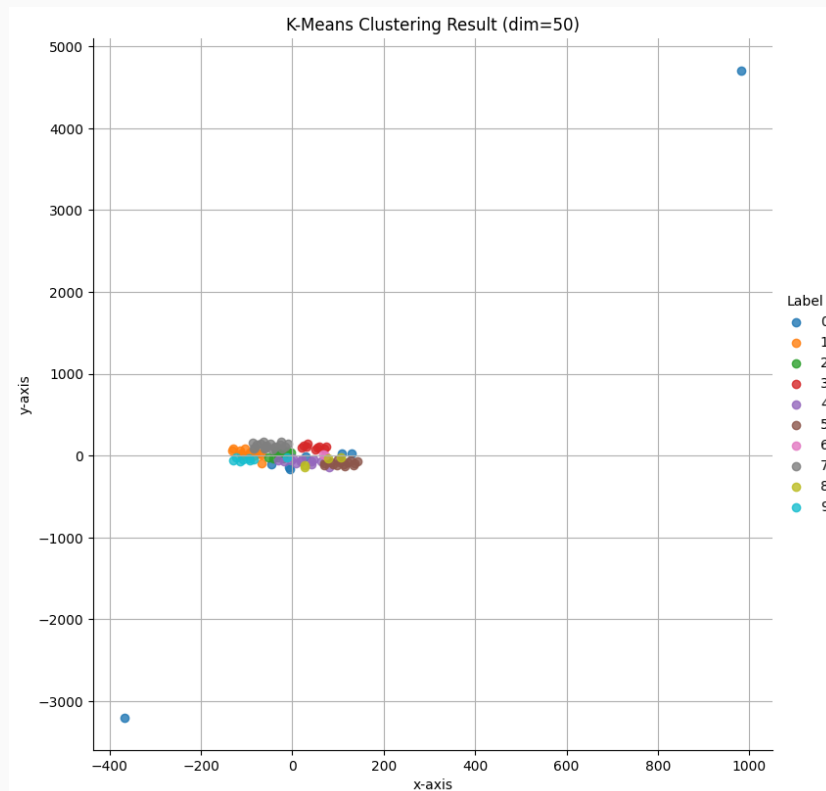
▶ K-means t-SNE 시각화 결과 (dim=784)



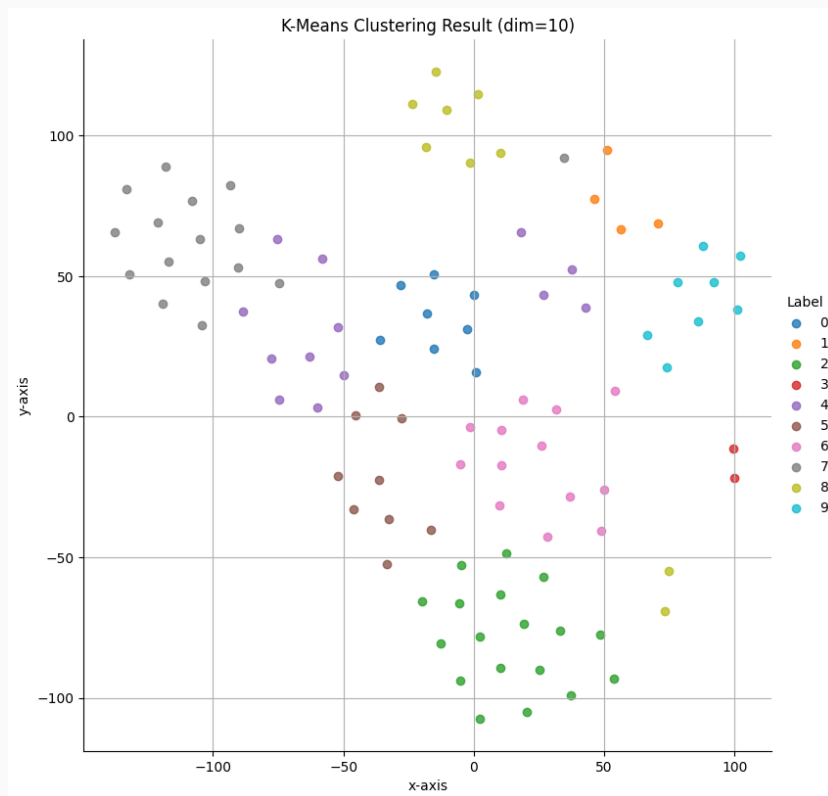
▶ K-means t-SNE 시각화 결과 (dim=100)



▶ K-means t-SNE 시각화 결과 (dim=50)



► K-means t-SNE 시각화 결과 (dim=10)



4.2 DBSCAN 결과

DBSCAN 클러스터링 결과 및 ARI 점수는 아래와 같다.

차원 (Dimension)	클러스터 수	ARI 점수
784 (원본)	1	0.0000
100	8	0.0070
50	19	0.0200
10	29	0.1722

▶ DBSCAN ARI 결과 화면

```
Dim= 784 | eps=10 | clusters=1 | noise=0 | ARI=0.0000
Dim= 100 | eps=4 | clusters=8 | noise=1621 | ARI=0.0070
Dim=  50 | eps=3 | clusters=19 | noise=2708 | ARI=0.0200
Dim=  10 | eps=1.5 | clusters=29 | noise=2417 | ARI=0.1722
```

```
===== DBSCAN ARI 결과 =====
Dimension    Clusters    ARI
-----
784          1          0.0000
100          8          0.0070
50           19          0.0200
10           29          0.1722
```

▶ DBSCAN t-SNE 시각화 코드

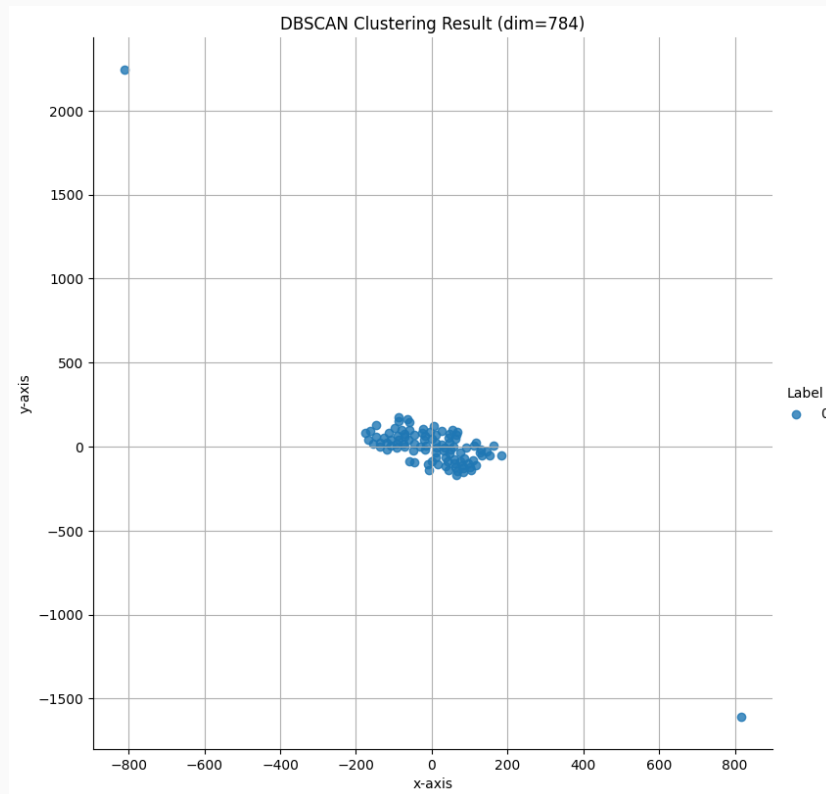
```
for dim in [784, 100, 50, 10]:
    # 100 개 샘플링 (같은 sample_idx 사용)
    X_sample = pca_data[dim][sample_idx]
    cluster_sample = dbscan_labels[dim][sample_idx]

    # t-SNE
    tsne = TSNE(n_components=2, learning_rate=300, perplexity=30,
                early_exaggeration=12, init='random', random_state=42)
    X_tsne = pd.DataFrame(tsne.fit_transform(X_sample))

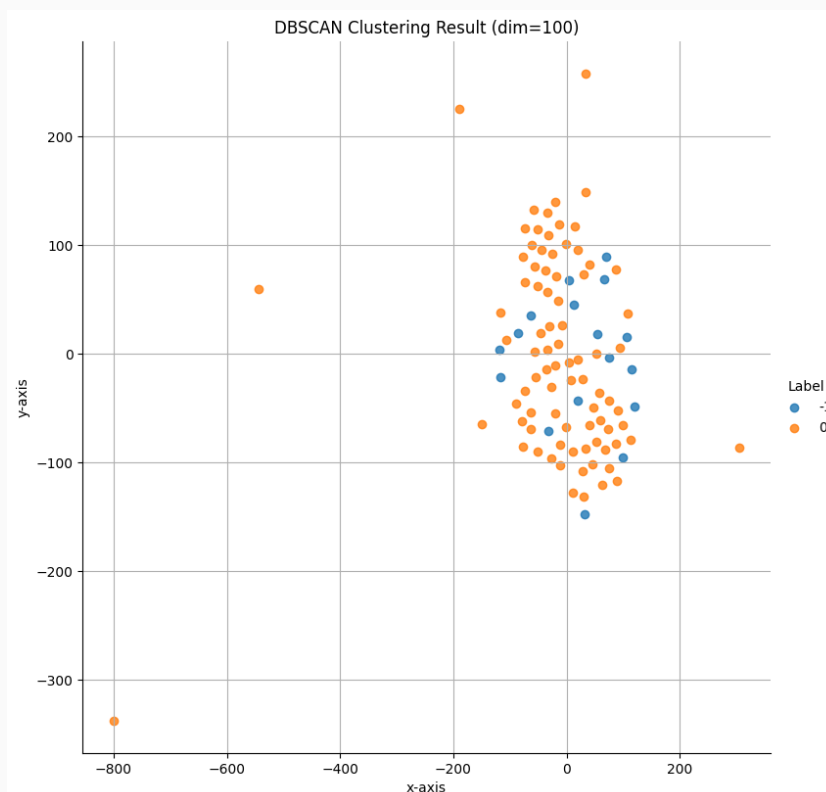
    # DataFrame 구성
    testDF = pd.DataFrame(data=X_tsne.values, columns=["x-axis", "y-axis"])
    testDF = pd.concat((testDF, pd.Series(cluster_sample, name="Label")), axis=1)

    # 시각화
    sns.lmplot(x="x-axis", y="y-axis", hue="Label", data=testDF, fit_reg=False,
               height=8)
    plt.title(f"DBSCAN Clustering Result (dim={dim})")
    plt.grid()
    plt.show()
```

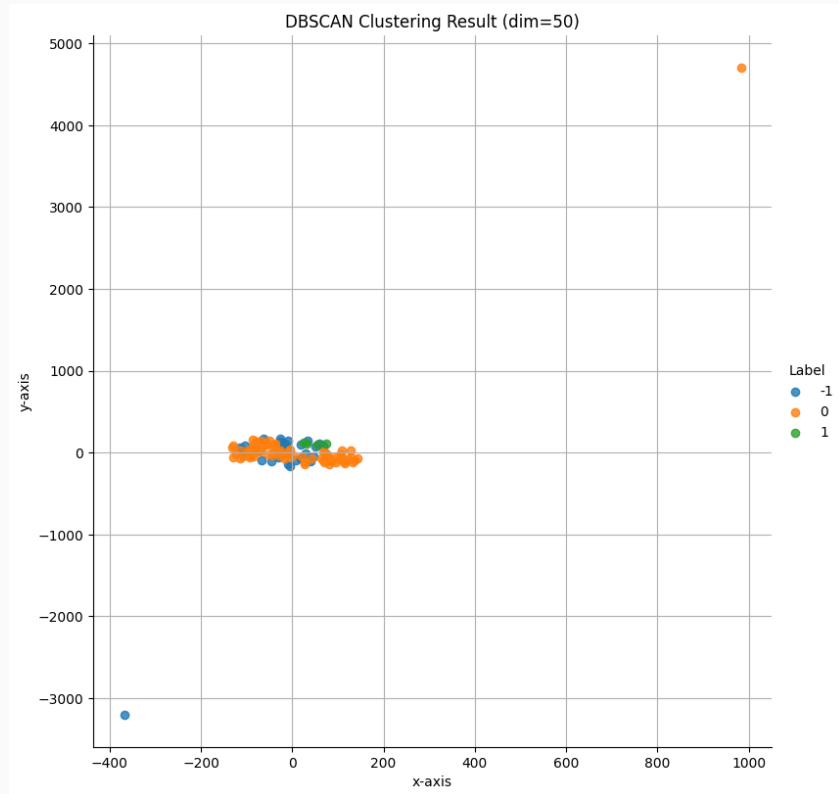
► DBSCAN t-SNE 시각화 결과 (dim=784)



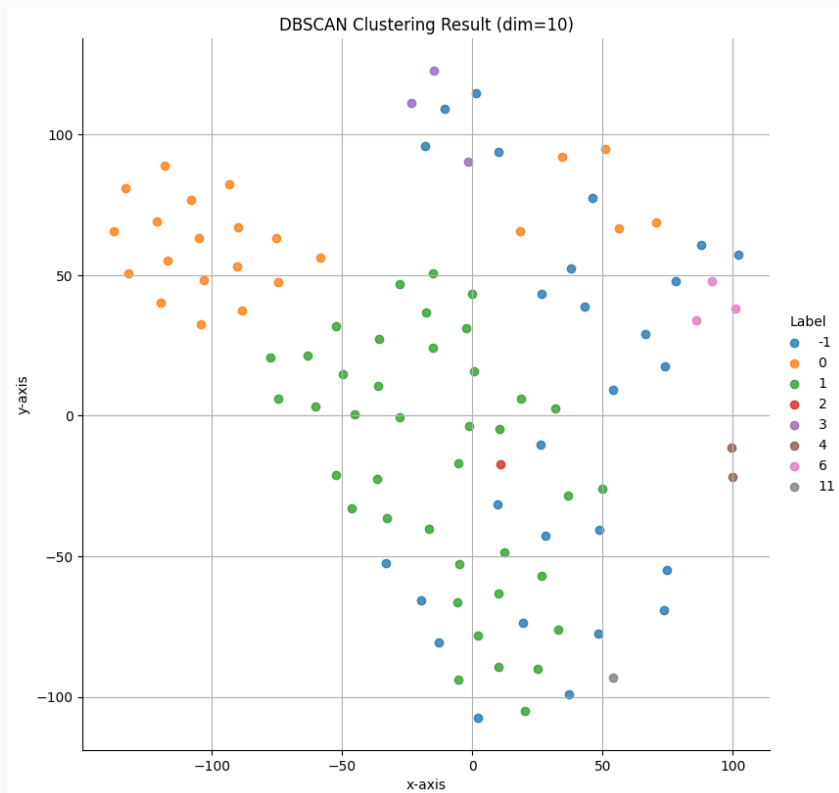
► DBSCAN t-SNE 시각화 결과 (dim=100)



► DBSCAN t-SNE 시각화 결과 (dim=50)



► DBSCAN t-SNE 시각화 결과 (dim=10)



5. 결과 분석 (Analysis)

5.1 K-means 분석

(1) PCA 차원 축소 효과

K-means는 784, 100, 50, 10차원 모두에서 유사한 ARI(0.34~0.35)를 보였다. 이는 PCA로 차원을 줄여도 클러스터링 성능이 크게 저하되지 않음을 의미하며, PCA가 중요한 특징을 효과적으로 보존하고 있음을 시사한다.

(2) t-SNE 시각화

차원이 낮을수록 t-SNE 시각화에서 클러스터가 더 명확하게 분리되어 나타났다. 784차원과 100차원에서는 100개 샘플이 고차원 공간에서 뭉쳐 보이는 반면, 10차원에서는 각 클래스가 비교적 분리된 형태로 시각화되었다.

5.2 DBSCAN 분석

(1) 고차원에서의 한계

DBSCAN은 784차원에서 클러스터 1개만 생성하며 ARI 0.0000을 기록하였다. 이는 고차원 공간에서 유클리드 거리 기반의 밀도 추정이 어렵기 때문이다(차원의 저주, Curse of Dimensionality).

(2) 차원 축소 후 성능

차원이 낮아질수록 ARI가 증가하는 경향을 보였다. 10차원에서 ARI 0.1722로 가장 높은 성능을 보였으나, K-means(0.35)에 비해 여전히 낮다. 이는 Fashion MNIST 데이터의 클러스터 경계가 밀도 기반보다는 중심 기반으로 더 잘 구분되기 때문이다.

5.3 K-means vs DBSCAN 종합 비교

알고리즘	최고 ARI	최적 차원	특징
K-means	0.3516	50차원	안정적인 성능, 차원 변화에 강건
DBSCAN	0.1722	10차원	고차원에서 취약, 저차원에서 개선

전체적으로 K-means가 DBSCAN보다 Fashion MNIST 데이터에 더 적합한 클러스터링 알고리즘임을 확인할 수 있었다. DBSCAN은 노이즈에 강하고 비구형 클러스터를 감지할 수 있다는 장점이 있으나, 고차원 이미지 데이터에서는 적절한 eps 설정이 어렵고 성능이 낮게 나타났다.